MAJOR PROJECT REPORT

Tester: Sravan Kumar Mandeti, Amritha Sree, Akshata

Testing Environment: Windows

3. API Security Testing Framework

Objective: Automated API Security Testing Framework using Flask, Python, Postman, OWASP ZAP, and Burp Suite.

---

1. Introduction

In this project, I developed a basic API Security Testing Framework to identify common security vulnerabilities in RESTful APIs. The aim was to automate the detection of common flaws like BOLA, Rate-limiting bypass, and API key exposure using Python scripting and validate the API's security using popular testing tools such as Postman, OWASP ZAP, and Burp Suite.

---

2. Tools & Technologies Used

Tool	Purpose	Official Link
Python (3.10+)	Programming language used for API server and scanner scripts	https://www.python.org/downloads/
Postman	Tool for manual API testing	https://www.postman.com/downloads/
OWASP ZAP	Automated security scanner for APIs/web apps	https://www.zaproxy.org/download/

Tool	Purpose	Official Link
Burp Suite (Community Edition)	**Manual web security testing tool**	**https://s9.up4ever.download:8443/d/ahojv7s2pqy52ag4kzlakxz3fxoxzwjicpu4ozn7c3q2h7mxwpkmosme5zxzgfyznfttp5x6/Burp%20Suite%20Professional%20Edition%20v2025.6.1%20x64%20Full%20Activated%20+%20Extensions%20-%20WwW.Dr-FarFar.CoM.zip** **(use the add blocker to download this file)** **(for running this, we need to have the latest JDK)**
Java Runtime Environment (JRE)	**Required to run ZAP**	**https://www.oracle.com/java/technologies/javase-jre8-downloads.html** ***(or use OpenJDK)***

3. Project Structure

```
api-security-framework/
├── api_server/
│   ├── app.py
│   └── requirements.txt
├── scanner/
│   └── security_scanner.py
```

- ➔ This is the overall structure of the project, this we did with a .zip file then extracted it and then worked on with the code (.py file, requirements and all)

- ➔ Here we ran the code using **cmd**. See below how we did (for adding one more terminal use arrow down button it will show option of the terminal click on the cmd only)

```
C:\Windows\System32\cmd.e   Command Prompt

Microsoft Windows [Version 10.0.26100.4484]
(c) Microsoft Corporation. All rights reserved.

C:\>cd api-security-framework

C:\api-security-framework>python -m venv venv

C:\api-security-framework>venv\Scripts\activate

(venv) C:\api-security-framework>pip install -r requirements.txt
Requirement already satisfied: flask_limiter in c:\api-security-framework\venv\lib\site-packages (from -r requirements.txt (line 2)) (3.12)
Requirement already satisfied: flask in c:\api-security-framework\venv\lib\site-packages (from -r requirements.txt (line 3)) (3.1.1)
Requirement already satisfied: pyjwt in c:\api-security-framework\venv\lib\site-packages (from -r requirements.txt (line 4)) (2.10.1)
Requirement already satisfied: requests in c:\api-security-framework\venv\lib\site-packages (from -r requirements.txt (line 5)) (2.32.4)
Requirement already satisfied: flasgger in c:\api-security-framework\venv\lib\site-packages (from -r requirements.txt (line 6)) (0.9.7.1)
Requirement already satisfied: limits>=3.13 in c:\api-security-framework\venv\lib\site-packages (from flask_limiter->-r requirements.txt (line 2)) (5.4.0)
Requirement already satisfied: ordered-set<5,>4 in c:\api-security-framework\venv\lib\site-packages (from flask_limiter->-r requirements.txt (line 2)) (4.1.0)
Requirement already satisfied: rich<14,>=12 in c:\api-security-framework\venv\lib\site-packages (from flask_limiter->-r requirements.txt (line 2)) (13.9.4)
Requirement already satisfied: markdown-it-py>=2.2.0 in c:\api-security-framework\venv\lib\site-packages (from rich<14,>=12->flask_limiter->-r requirements.txt (line 2)) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in c:\api-security-framework\venv\lib\site-packages (from rich<14,>=12->flask_limiter->-r requirements.txt (line 2)) (2.19.2)
Requirement already satisfied: blinker>=1.9.0 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (1.9.0)
Requirement already satisfied: click>=8.1.3 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (8.2.1)
Requirement already satisfied: itsdangerous>=2.2.0 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (2.2.0)
Requirement already satisfied: jinja2>=3.1.2 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (3.1.6)
Requirement already satisfied: markupsafe>=2.1.1 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (3.0.2)
Requirement already satisfied: werkzeug>=3.1.0 in c:\api-security-framework\venv\lib\site-packages (from flask->-r requirements.txt (line 3)) (3.1.3)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\api-security-framework\venv\lib\site-packages (from requests->-r requirements.txt (line 5)) (3.4.2)
Requirement already satisfied: idna<4,>=2.5 in c:\api-security-framework\venv\lib\site-packages (from requests->-r requirements.txt (line 5)) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\api-security-framework\venv\lib\site-packages (from requests->-r requirements.txt (line 5)) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\api-security-framework\venv\lib\site-packages (from requests->-r requirements.txt (line 5)) (2025.6.15)
Requirement already satisfied: PyYAML>=3.0 in c:\api-security-framework\venv\lib\site-packages (from flasgger->-r requirements.txt (line 6)) (6.0.2)
Requirement already satisfied: jsonschema>=3.0.1 in c:\api-security-framework\venv\lib\site-packages (from flasgger->-r requirements.txt (line 6)) (4.24.0)
Requirement already satisfied: mistune in c:\api-security-framework\venv\lib\site-packages (from flasgger->-r requirements.txt (line 6)) (3.1.3)
Requirement already satisfied: six>=1.10.0 in c:\api-security-framework\venv\lib\site-packages (from flasgger->-r requirements.txt (line 6)) (1.17.0)
Requirement already satisfied: packaging in c:\api-security-framework\venv\lib\site-packages (from flasgger->-r requirements.txt (line 6)) (25.0)
Requirement already satisfied: colorama in c:\api-security-framework\venv\lib\site-packages (from click>=8.1.3->flask->-r requirements.txt (line 3)) (0.4.6)
Requirement already satisfied: attrs>=22.2.0 in c:\api-security-framework\venv\lib\site-packages (from jsonschema>=3.0.1->flasgger->-r requirements.txt (line 6)) (25.3.0)
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in c:\api-security-framework\venv\lib\site-packages (from jsonschema>=3.0.1->flasgger->-r requirements.txt (line 6)) (2025.4.1)
Requirement already satisfied: referencing>=0.28.4 in c:\api-security-framework\venv\lib\site-packages (from jsonschema>=3.0.1->flasgger->-r requirements.txt (line 6)) (0.36.2)
Requirement already satisfied: rpds-py>=0.7.1 in c:\api-security-framework\venv\lib\site-packages (from jsonschema>=3.0.1->flasgger->-r requirements.txt (line 6)) (0.26.0)
Requirement already satisfied: deprecated>=1.2 in c:\api-security-framework\venv\lib\site-packages (from limits>=3.13->flask_limiter->-r requirements.txt (line 2)) (1.2.18)
Requirement already satisfied: typing_extensions in c:\api-security-framework\venv\lib\site-packages (from limits>=3.13->flask_limiter->-r requirements.txt (line 2)) (4.14.0)
Requirement already satisfied: wrapt<2,>=1.10 in c:\api-security-framework\venv\lib\site-packages (from deprecated>=1.2->limits>=3.13->flask_limiter->-r requirements.txt (line 2)) (1.17.2)
Requirement already satisfied: mdurl~=0.1 in c:\api-security-framework\venv\lib\site-packages (from markdown-it-py>=2.2.0->rich<14,>=12->flask_limiter->-r requirements.txt (line 2)) (0.1.2)

(venv) C:\api-security-framework>cd api_server

(venv) C:\api-security-framework\api_server>python app.py
C:\api-security-framework\venv\Lib\site-packages\flask_limiter\extension.py:324: UserWarning: Using the in-memory storage for tracking rate limits as no storage was explicitly specified. This is not recommend
ed for production use. See: https://flask-limiter.readthedocs.io#configuring-a-storage-backend for documentation about configuring the storage backend.
  warnings.warn(
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

```
Extremely humid
Now
Search
5:37 AM
7/4/2025
```

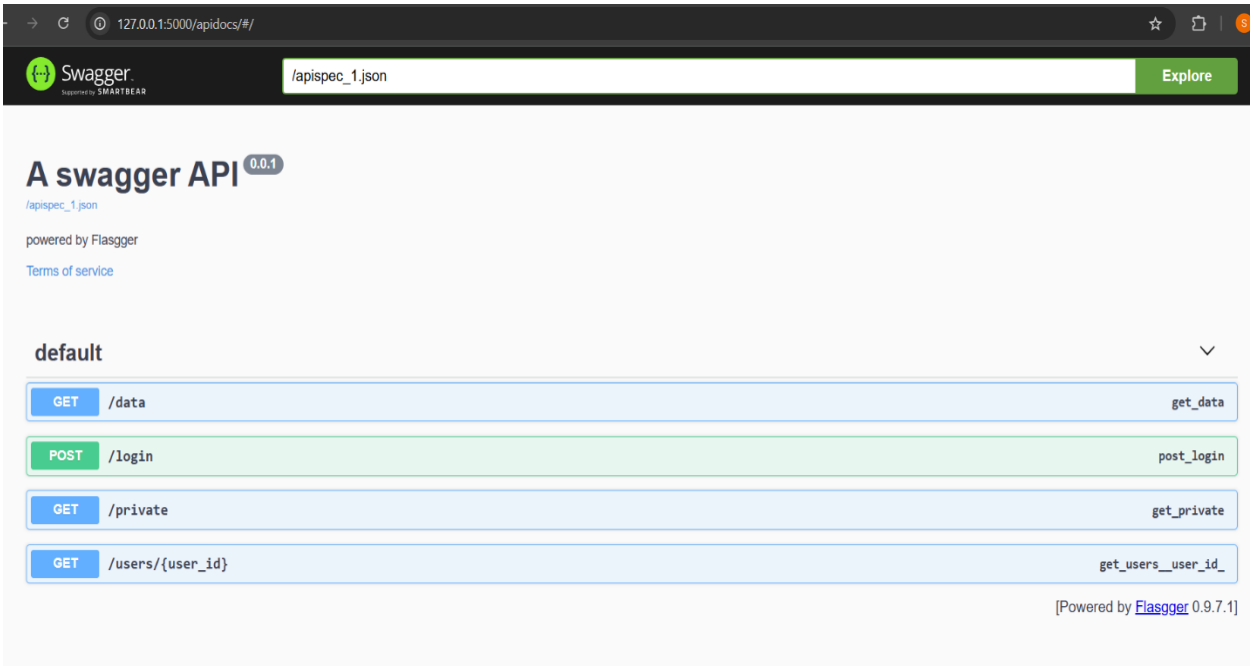
```
C:\Windows\System32\cmd.ex   Command Prompt

C:\>cd C:\api-security-framework

C:\api-security-framework>venv\Scripts\activate

(venv) C:\api-security-framework>cd scanner

(venv) C:\api-security-framework\scanner>python security_scanner.py
[*] Waiting for API to be up... API is up!
[*] Token received: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJleHAiOjE3NTE1ODIyMTl9.heH4ADqcpGdStKsquFNm8J
VCj_vBeVmnaqgSz4_3Ih0
[X] BOLA vulnerability detected!
[*] Sending spoofed requests to /data to bypass rate limit...
Attempt 1 (IP 192.168.0.0): 200
Attempt 2 (IP 192.168.0.1): 200
Attempt 3 (IP 192.168.0.2): 200
Attempt 4 (IP 192.168.0.3): 200
Attempt 5 (IP 192.168.0.4): 200
Attempt 6 (IP 192.168.0.5): 429
[✓] Rate limiting is enforced!
[✓] API key is secure.

(venv) C:\api-security-framework\scanner>python security_scanner.py
[*] Waiting for API to be up... API is up!
[*] Token received: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJleHAiOjE3NTE1ODg2NTd9.YvBjDoLXdlyKGm56P02T6q
VPXIwcYGF_R7zCBEV2a8s
[✓] BOLA protection is in place.
[*] Sending spoofed requests to /data to bypass rate limit...
Attempt 1 (IP 192.168.0.0): 200
Attempt 2 (IP 192.168.0.1): 200
Attempt 3 (IP 192.168.0.2): 200
Attempt 4 (IP 192.168.0.3): 200
Attempt 5 (IP 192.168.0.4): 200
Attempt 6 (IP 192.168.0.5): 429
[✓] Rate limiting is enforced!
[✓] API key is secure.
```

➔ Here one thing you need to remember first need to run the app.py and then need to run the security_scanner.py(in the both terminals once app.py execution done)

➔ After running the Python security_scanner.py you will be redirected to the browser there you can find the results of your code. Check given below screen short for reference

API Security Testing Server

Use Postman or scanner.py to test endpoints.



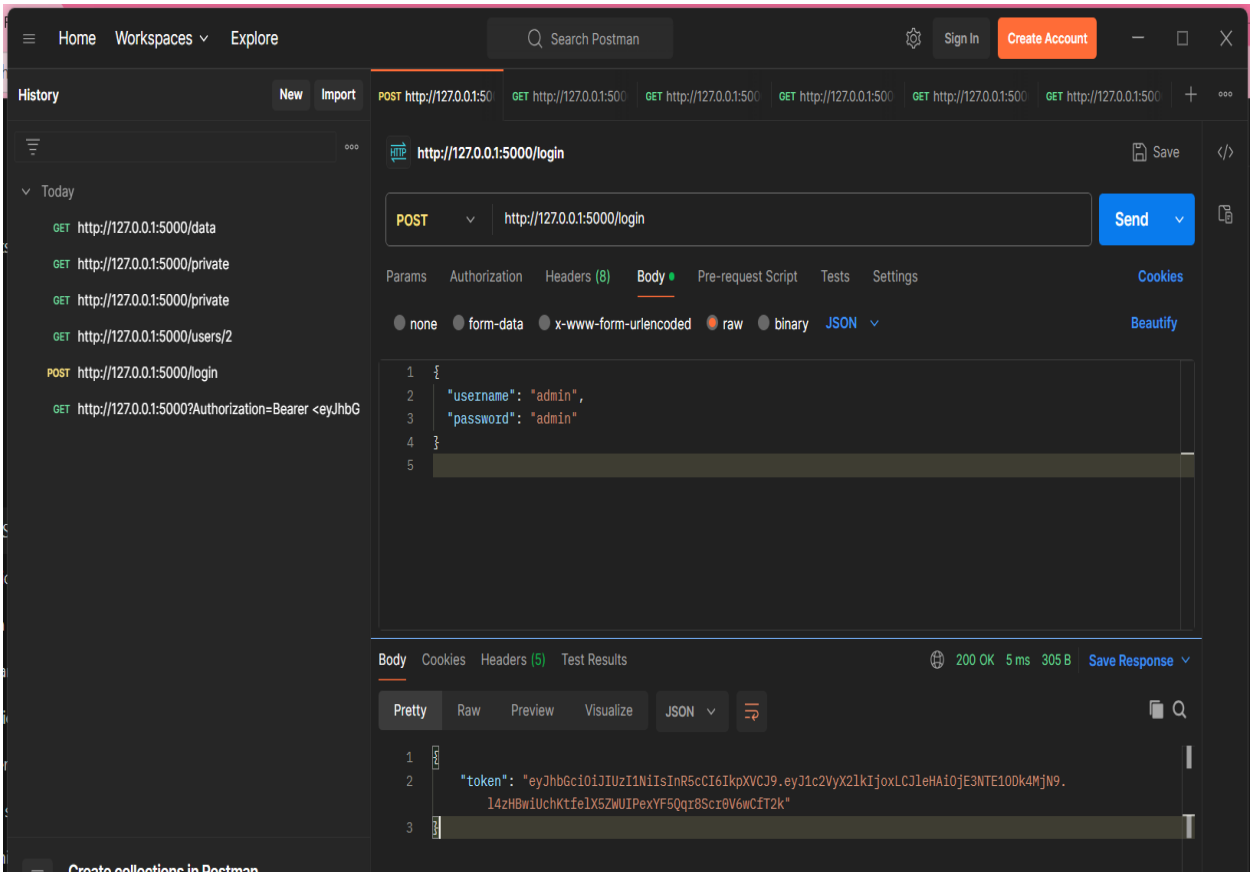
4. Step-by-Step Implementation

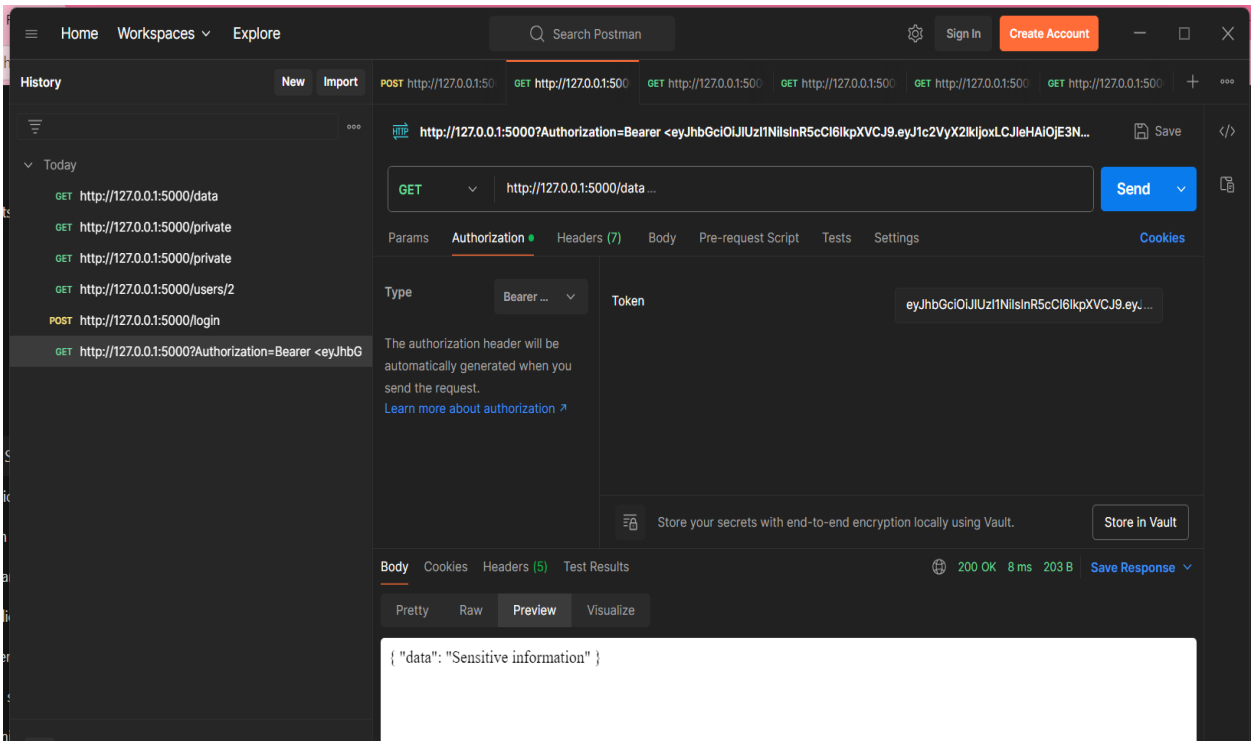
Step 1: Flask API Setup

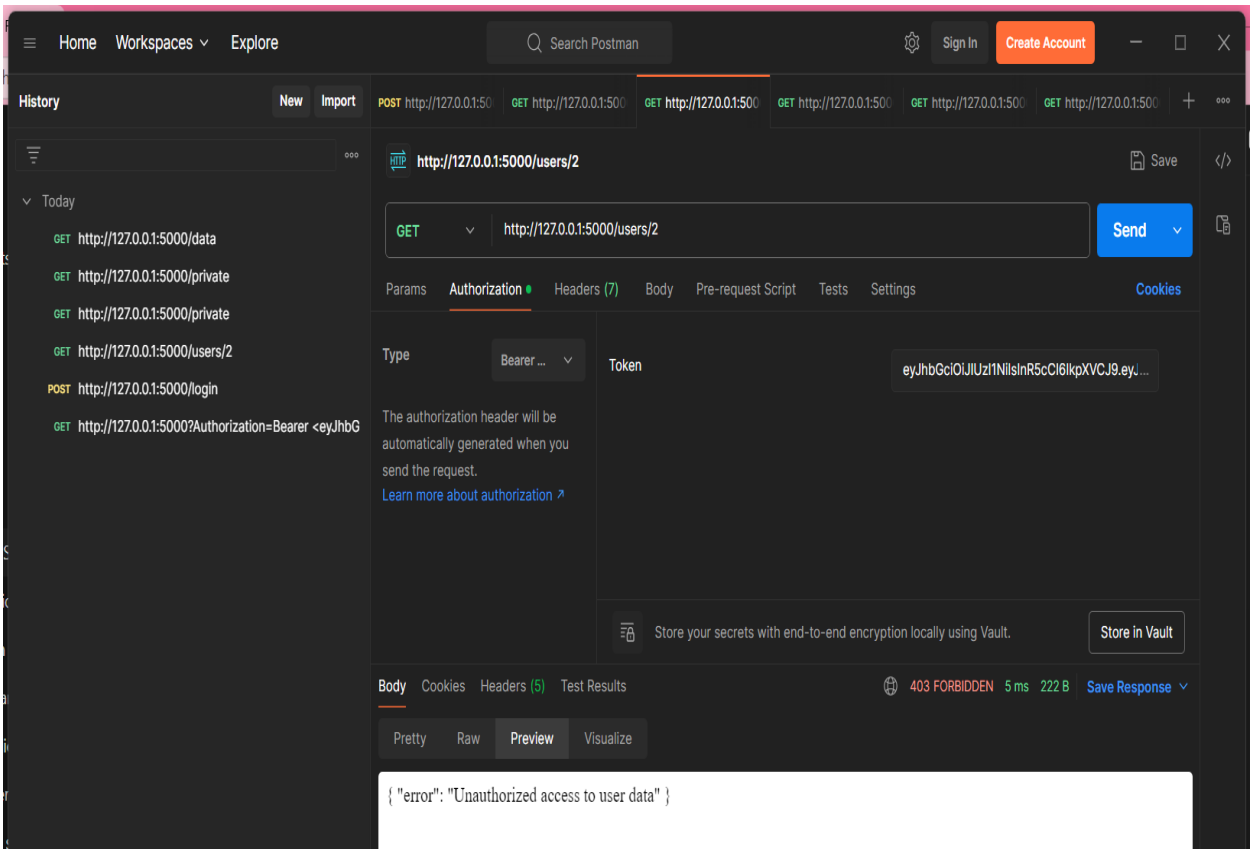
- Created a Flask API with /login and /data endpoints.
- Implemented token-based JWT authentication.
- Used Flask-Limiter to enforce rate limits (5 requests/min).
- Dummy user credentials: admin / admin.

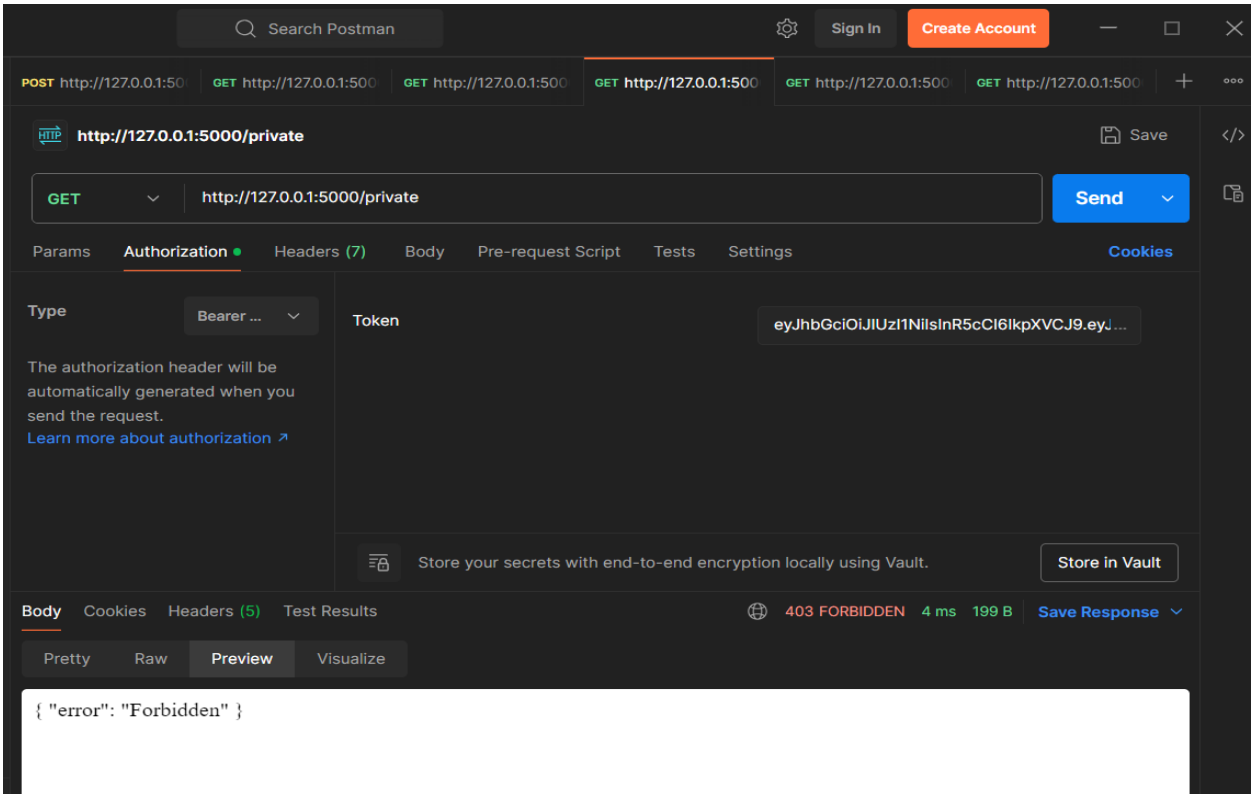
Endpoints Developed:

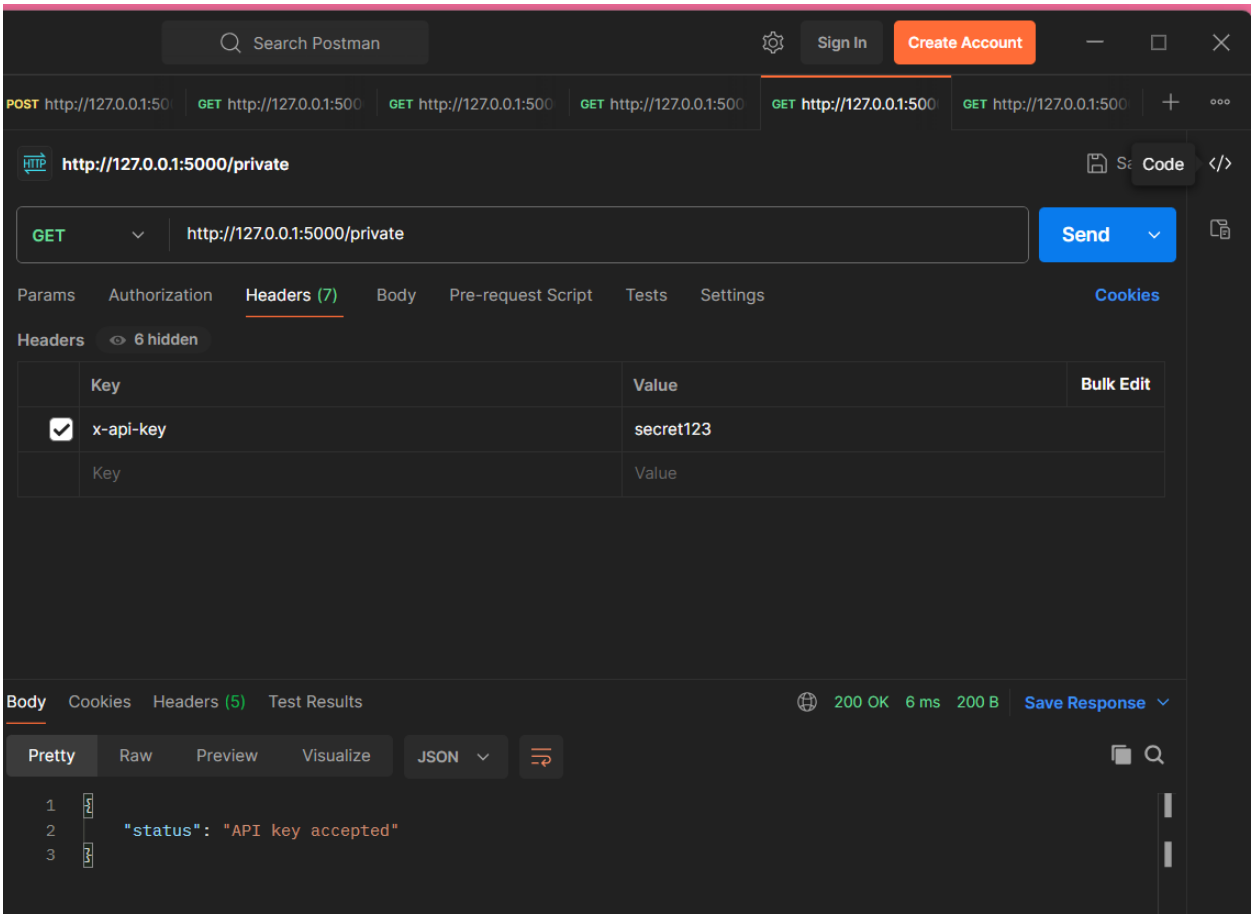
- POST /login – returns a JWT token.
- GET /data – protected data endpoint.

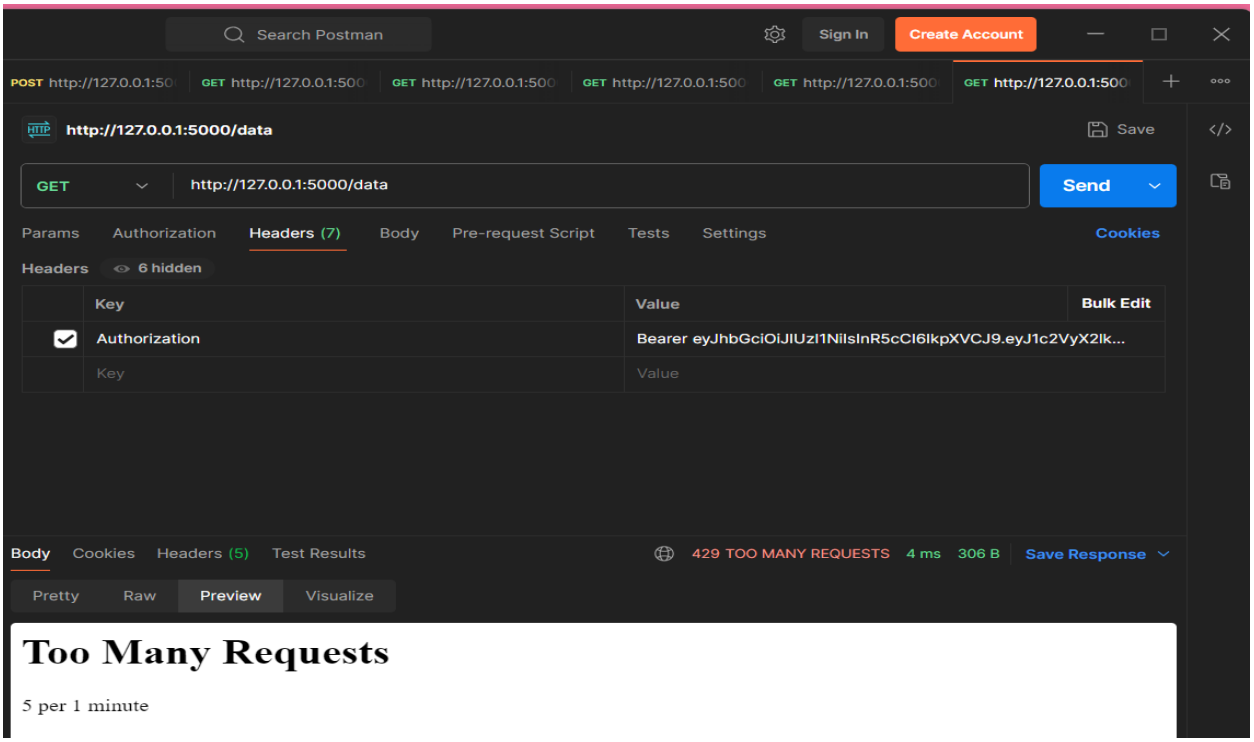



Home
Workspaces
Explore
Search Postman
Sign In
Create Account
History
New
Import
Today
GET http://127.0.0.1:5000/data
GET http://127.0.0.1:5000/private
GET http://127.0.0.1:5000/private
GET http://127.0.0.1:5000/users/2
POST http://127.0.0.1:5000/login
GET http://127.0.0.1:5000?Authorization=Bearer <eyJhbG
http://127.0.0.1:5000?Authorization=Bearer <eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2IkIjoxLCJleHAiOjE3N...
Save
GET
http://127.0.0.1:5000/data ...
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Type
Bearer ...
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ....
The authorization header will be automatically generated when you send the request.
Learn more about authorization ↗
Store your secrets with end-to-end encryption locally using Vault.
Store in Vault
Body
Cookies
Headers (5)
Test Results
200 OK
8 ms
203 B
Save Response
Pretty
Raw
Preview
Visualize
{ "data": "Sensitive information" }


Home
Workspaces
Explore
Search Postman
Sign In
Create Account
History
New
Import
Today
GET http://127.0.0.1:5000/data
GET http://127.0.0.1:5000/private
GET http://127.0.0.1:5000/private
GET http://127.0.0.1:5000/users/2
POST http://127.0.0.1:5000/login
GET http://127.0.0.1:5000?Authorization=Bearer <eyJhbG
http://127.0.0.1:5000/users/2
Save
GET
http://127.0.0.1:5000/users/2
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Type
Bearer ...
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ....
The authorization header will be automatically generated when you send the request.
Learn more about authorization ↗
Store your secrets with end-to-end encryption locally using Vault.
Store in Vault
Body
Cookies
Headers (5)
Test Results
403 FORBIDDEN
5 ms
222 B
Save Response
Pretty
Raw
Preview
Visualize
{ "error": "Unauthorized access to user data" }


Search Postman
Sign In
Create Account
POST http://127.0.0.1:50
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
http://127.0.0.1:5000/private
Save
GET
http://127.0.0.1:5000/private
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Type
Bearer ...
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.ey...
The authorization header will be automatically generated when you send the request.
Learn more about authorization
Store your secrets with end-to-end encryption locally using Vault.
Store in Vault
Body
Cookies
Headers (5)
Test Results
403 FORBIDDEN
4 ms
199 B
Save Response
Pretty
Raw
Preview
Visualize
{ "error": "Forbidden" }


Search Postman
Sign In
Create Account
POST http://127.0.0.1:50
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
GET http://127.0.0.1:500
http://127.0.0.1:5000/private
Code
GET
http://127.0.0.1:5000/private
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Headers
6 hidden
Key
Value
Bulk Edit
x-api-key
secret123
Key
Value
Body
Cookies
Headers (5)
Test Results
200 OK
6 ms
200 B
Save Response
Pretty
Raw
Preview
Visualize
JSON
{
"status": "API key accepted"
}



Step 2: Automated Scanner Script

Developed security_scanner.py that:

- Waits until the API is live.
- Authenticates with /login and gets JWT token.
- Checks for BOLA vulnerability.
- Spoofs IP headers to test rate-limiting bypass.
- Check whether API keys are exposed or accepted by the server.

Results:

[✓] API is up!

[✓] Token received

[✓] BOLA protection is in place

[✓] Rate limiting is enforced

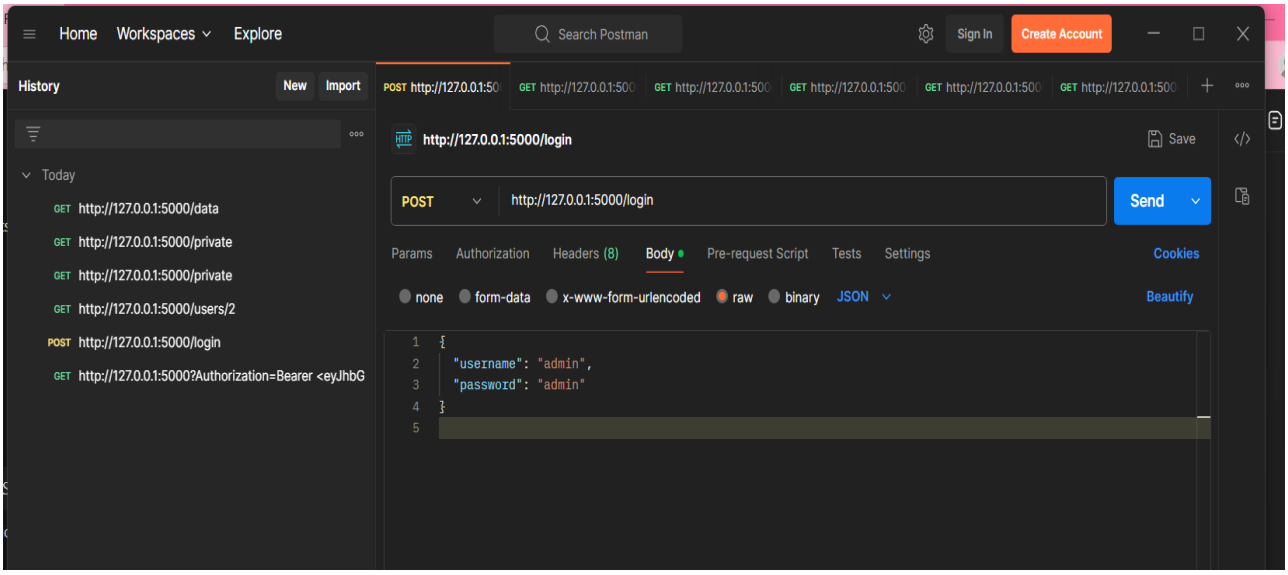
[✓] API key is not exposed

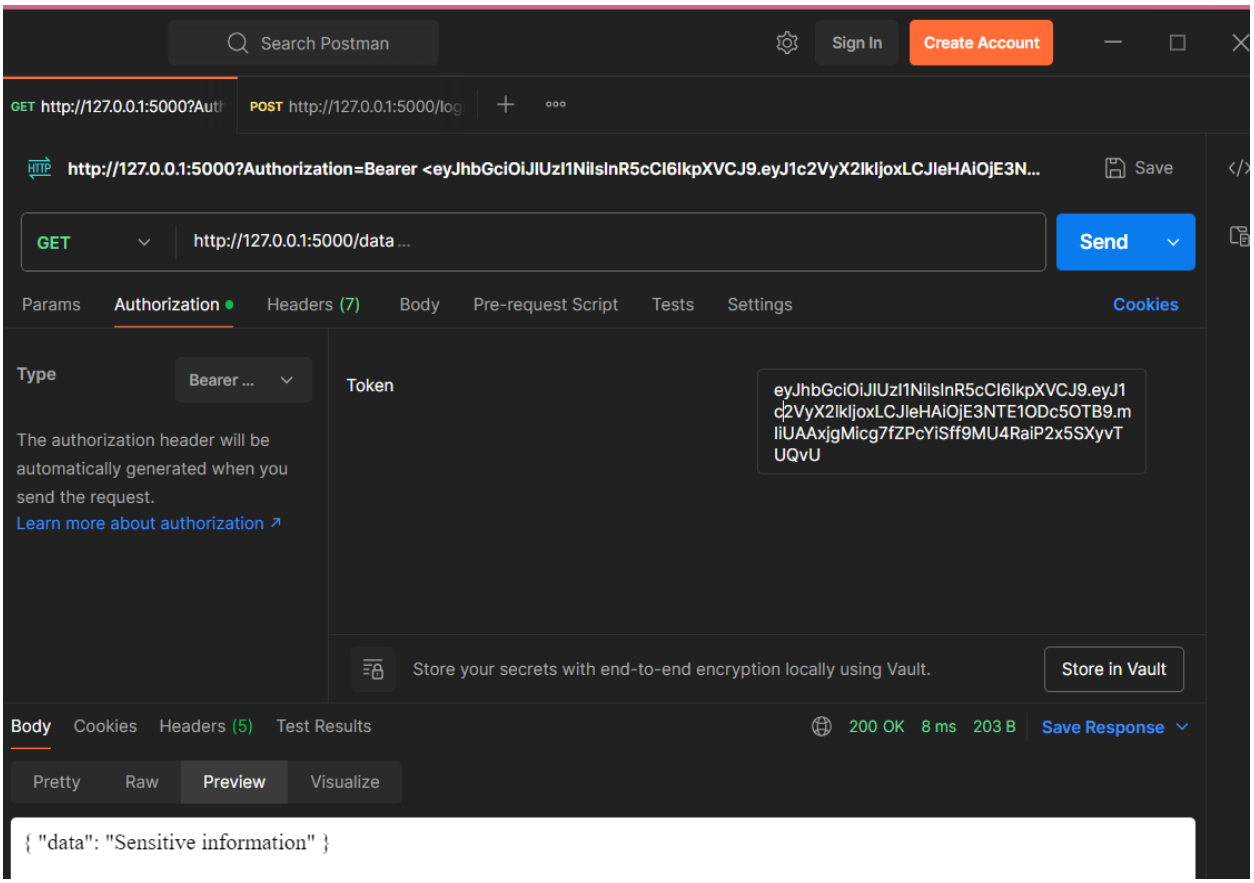
```
C:\Windows\System32\cmd.ex   Command Prompt

C:\>cd C:\api-security-framework

C:\api-security-framework>venv\Scripts\activate

(venv) C:\api-security-framework>cd scanner

(venv) C:\api-security-framework\scanner>python security_scanner.py
[*] Waiting for API to be up... API is up!
[*] Token received: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJleHAiOjE3NTE1ODIyMTl9.heH4ADqcpGdStKsquFNm8J
VCj_vBeVmnaqgSz4_3Ih0
[X] BOLA vulnerability detected!
[*] Sending spoofed requests to /data to bypass rate limit...
Attempt 1 (IP 192.168.0.0): 200
Attempt 2 (IP 192.168.0.1): 200
Attempt 3 (IP 192.168.0.2): 200
Attempt 4 (IP 192.168.0.3): 200
Attempt 5 (IP 192.168.0.4): 200
Attempt 6 (IP 192.168.0.5): 429
[✓] Rate limiting is enforced!
[✓] API key is secure.

(venv) C:\api-security-framework\scanner>python security_scanner.py
[*] Waiting for API to be up... API is up!
[*] Token received: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VyX2lkIjoxLCJleHAiOjE3NTE1ODg2NTd9.YvBjDoLXdlyKGm56P02T6q
VPXIwcYGF_R7zCBEV2a8s
[✓] BOLA protection is in place.
[*] Sending spoofed requests to /data to bypass rate limit...
Attempt 1 (IP 192.168.0.0): 200
Attempt 2 (IP 192.168.0.1): 200
Attempt 3 (IP 192.168.0.2): 200
Attempt 4 (IP 192.168.0.3): 200
Attempt 5 (IP 192.168.0.4): 200
Attempt 6 (IP 192.168.0.5): 429
[✓] Rate limiting is enforced!
[✓] API key is secure.
```

Step 3: Manual Testing with Postman

Performed the following:

- Sent POST /login with valid credentials and receive a JWT.
- Sent GET /data with and without JWT to confirm authorization works.
- Verified server responses: 200 OK for valid token, 401 Unauthorized otherwise.





Step 4: Vulnerability Scan using OWASP ZAP

- Setup local host URL: http://127.0.0.1:5000
- Ran Spider and Active Scan.
- Found 6 common alerts such as:
    - Missing CSP headers
    - Server header leaking info
    - Missing X-Content-Type-Options
- Verified that no critical vulnerabilities were detected in an active scan.

File Edit View Analyse Report Tools Import Export Online Help

Untitled Session - ZAP 2.16.1

Standard Mode

Sites

Quick Start | Request | Response | Requester

Contexts
Default Context
Sites

Automated Scan

ZAP by Checkmarx

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.

Please be aware that you should only attack applications that you have been specifically been given permission to test.

URL to attack: http://127.0.0.1:5000 Select...

Use traditional spider: ☑

Use ajax spider: If Modern with Firefox

Attack Stop

Progress: Attack complete - see the Alerts tab for details of any issues found

History | Search | Alerts | Output | Spider | Active Scan

Alerts (6)
- CSP: Failure to Define Directive with No Fallback (3)
- Content Security Policy (CSP) Header Not Set (3)
  - GET: http://127.0.0.1:5000
  - GET: http://127.0.0.1:5000/robots.txt
  - GET: http://127.0.0.1:5000/sitemap.xml
- Missing Anti-clickjacking Header
  - GET: http://127.0.0.1:5000
- Server Leaks Version Information via "Server" HTTP Response Header Field (3)
  - GET: http://127.0.0.1:5000
  - GET: http://127.0.0.1:5000/robots.txt
  - GET: http://127.0.0.1:5000/sitemap.xml
- X-Content-Type-Options Header Missing
  - GET: http://127.0.0.1:5000
- User Agent Fuzzer (12)

Full details of any selected alert will be displayed here.

You can manually add alerts by right clicking on the relevant line in the history and selecting 'Add alert'.

You can also edit existing alerts by double clicking on them.

Alerts 0 3 2 1 Main Proxy: localhost:8080 Current Status

91°F Mostly cloudy Search 4:10 PM 7/4/2025

File Edit View Analyse Report Tools Import Export Online Help

Untitled Session - ZAP 2.16.1

Standard Mode

Sites

Quick Start | Request | Response | Requester

Contexts
Default Context
Sites
http://127.0.0.1:5000
GET:robots.txt
GET:sitemap.xml

Automated Scan

ZAP by Checkmarx

This screen allows you to launch an automated scan against an application - just enter its URL below and press 'Attack'.

Please be aware that you should only attack applications that you have been specifically been given permission to test.

URL to attack: http://127.0.0.1:5000 Select...

History | Search | Alerts | Output | Spider | Active Scan

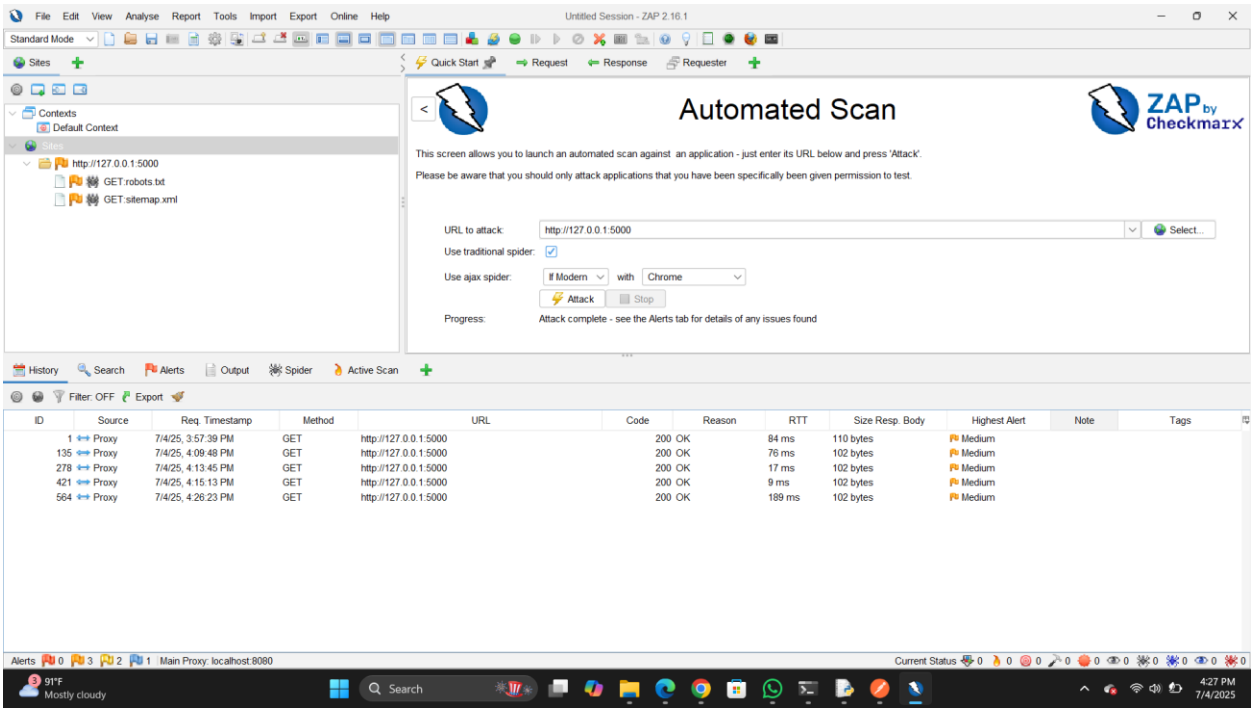
New Scan Progress: 4: http://127.0.0.1:5000 100% Current Scans: 0 URLs Found: 3 Nodes Added: 0 Export

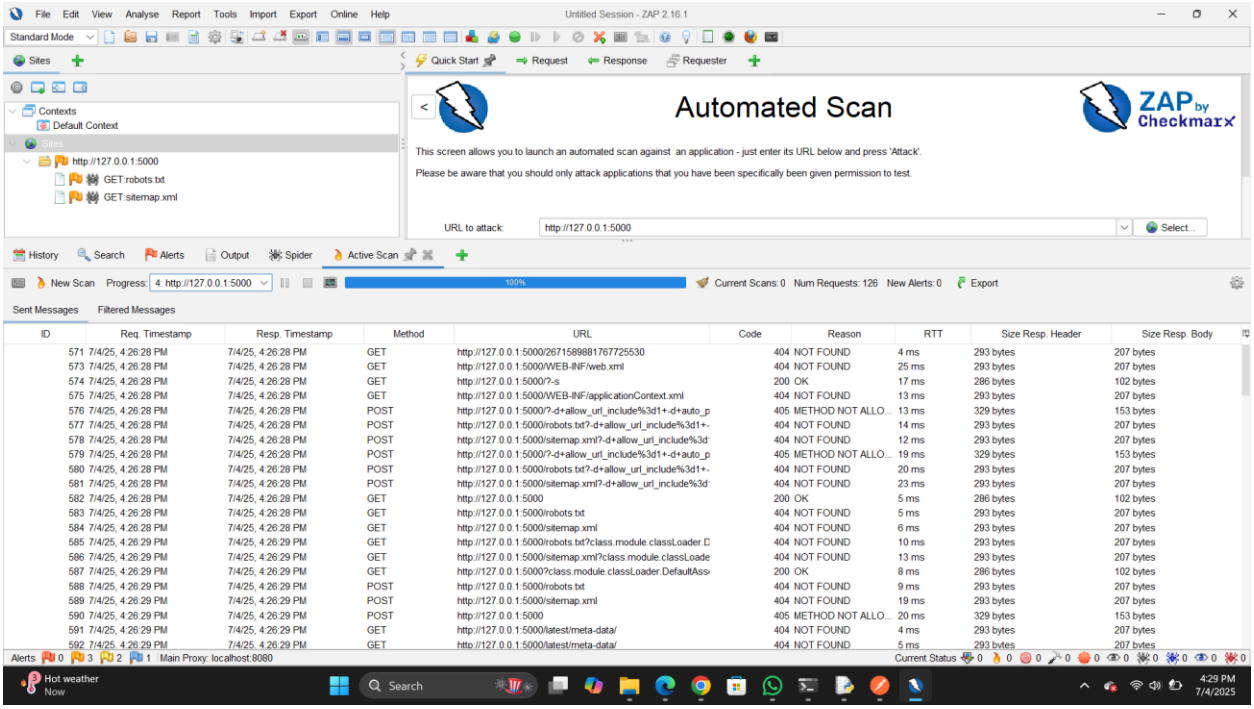
URLs | Added Nodes | Messages

Processed	Method	URI	Flags
	GET	http://127.0.0.1:5000	Seed
	GET	http://127.0.0.1:5000/robots.txt	Seed
	GET	http://127.0.0.1:5000/sitemap.xml	Seed

Alerts 0 3 2 1 Main Proxy: localhost:8080 Current Status

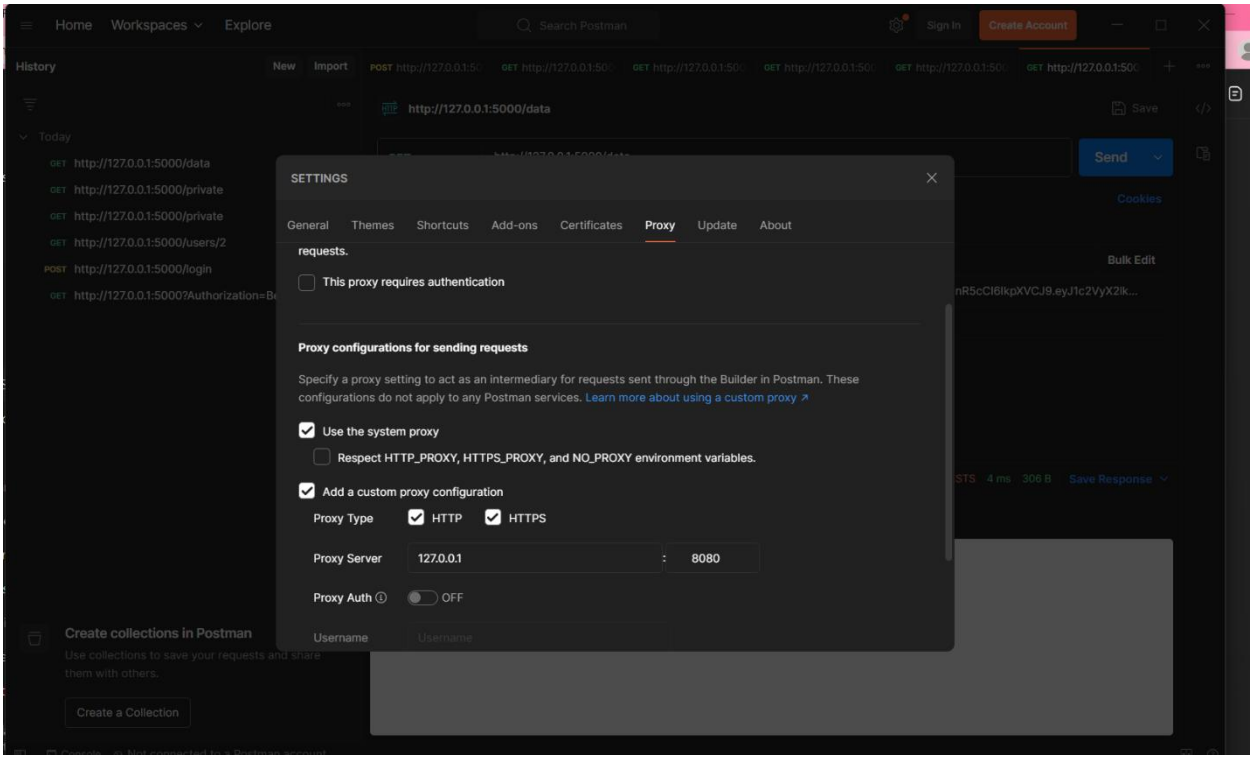
Hot weather Now Search 4:29 PM 7/4/2025



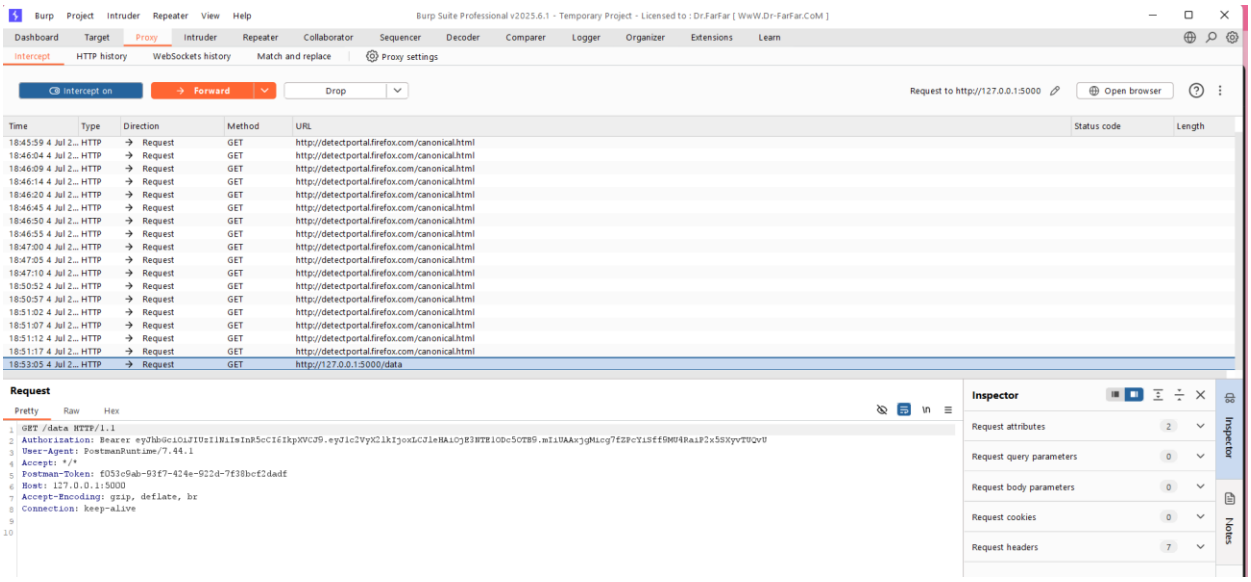


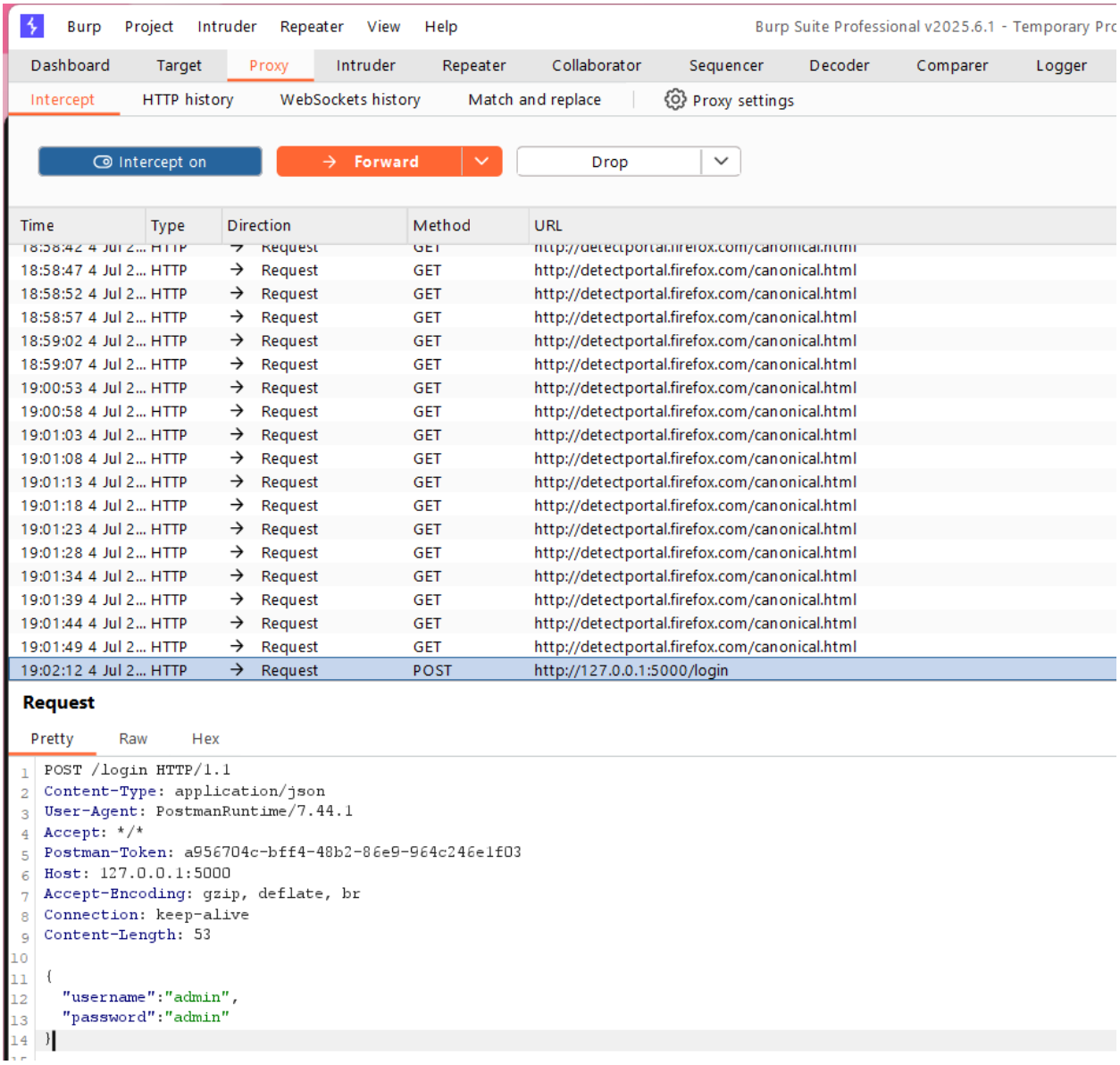
Step 5: Analysis using Burp Suite

(To make Postman requests visible in Burp Suite, I configured the global proxy settings in Postman to route traffic through Burp's proxy listener. I set the proxy server to 127.0.0.1 and the port to 8080, which matched the default proxy listener configuration in Burp Suite. This allowed me to intercept and inspect HTTP requests from Postman within Burp's Proxy → HTTP History tab.)



- Configured browser proxy to 127.0.0.1:8080.
- Intercepted requests to /login and /data.
- Observed:
    - JWT token passed in Authorization header.
    - Server responded with 200 OK for valid tokens.
- Successfully confirmed authentication mechanisms using Burp Suite.





---

5. Final Key Results

Vulnerability Tested	Status
API Availability	Confirmed
JWT Token Authentication	Working
BOLA Protection	Enforced

Vulnerability Tested	Status
Rate Limiting	Detected & enforced
API Key Exposure	Not Exposed
OWASP ZAP Alerts	Medium level headers
Burp Suite Token Intercept	Successful

---

6. Challenges Faced

- Flask rate limiter didn't redirect errors clearly; adjusted with IP spoofing to verify.
- Initial OWASP ZAP didn't launch due to JRE issue – resolved by locating Java path.
- Postman required manual token extraction for GET requests testing.
- Burp Suite required proper certificate setup and proxy configuration.

---

7. Conclusion

This project successfully demonstrates how to set up an automated API security testing framework that incorporates both scripting and professional tools. Through Flask, Python, Postman, OWASP ZAP, and Burp Suite, I was able to simulate real-world attacks and ensure that the API was resistant to some common threats.

This hands-on implementation helped reinforce my understanding of API security best practices.